(1) Brief description of prompt/uploaded material: I had given ChatGPT a brief description of our application, including what it does and how it works and also gave it my group's preferences in coding language for the backend system as Python as everyone is proficient in Python which will allow future work to be split equally.

(2)

hi chat, please give me a recommendation for a technology stack for my medium-lightweight mobile application. my application is a route recommendation system that generates routes for users based on their preferences and needs. the generated routes take into consideration accessibility needs for disabled persons and shaded routes for users who prefer so as well. we use LTA's OneMap API to feed in the live information for the route planning and the app generates a list of routes with the route that satisfies the user's request the best at the top. please generate a technology stack for this

Show more

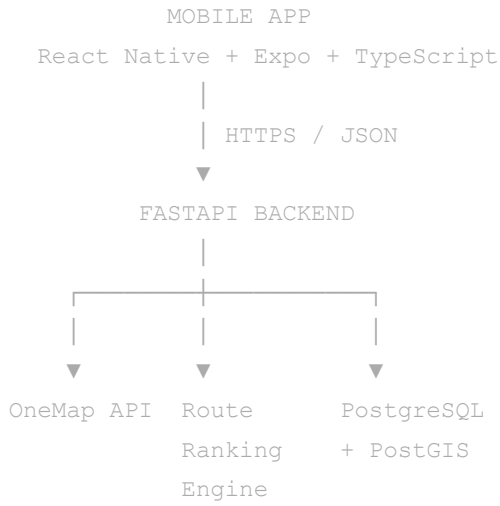
Worked for 18s

For your app, I'd recommend a fairly simple **3-layer architecture** rather than trying to build a complicated microservice setup. Your workload is mainly API calls, geospatial calculations, route scoring, user preferences, and map rendering, so you do not need an especially heavy stack.

Recommended stack

Layer	Recommendation	Main technologies
Frontend	React Native	**React Native + Expo + TypeScript + MapLibre**
Backend	Python REST API	**FastAPI + Pydantic + HTTPX + SQLAlchemy**
Database	Relational + geospatial DB	**PostgreSQL + PostGIS**
External data	Singapore map/routing	**OneMap APIs**
Deployment later	Managed hosting	Render/Railway/Fly.io + managed PostgreSQL

The overall architecture would look roughly like this:



1. Frontend — React Native + Expo + TypeScript

This would be my first choice for your project.

React Native lets you build both Android and iOS from essentially one codebase. **Expo** makes React Native development considerably easier by handling a lot of the native configuration and build process. Expo currently recommends starting projects using its standard `create-expo-app` setup. Expo docum... Use:

```
React Native
TypeScript
Expo
MapLibre React Native
Expo Router fetch /
Axios
```

I'd use **TypeScript rather than plain JavaScript**. For a group software project, the type checking becomes useful once you start passing around things like:

```
Route
UserPreferences
MobilityProfile
Location
AccessibilityRequirement
RouteScore
Hazard
```

For example, your frontend could have something conceptually like:

```ts
interface RoutePreference {
wheelchairAccessible: boolean;
preferShade: boolean;
avoidStairs: boolean;
maxWalkingDistance: number;
```

```
}

interface RouteResult {
routeId: string;     duration:
number;     distance: number;
accessibilityScore: number;
shadeScore: number;
overallScore: number;
}
```

That makes bugs caused by incorrect data structures much easier to catch.

Mapping: MapLibre

I'd use **MapLibre React Native** to display the actual map, route polylines, markers, accessibility information, hazards, etc. It supports Android and iOS and works with custom map styles/tiles. MapLibre +1

One caveat: MapLibre isn't available directly inside **Expo Go**; you'd need an Expo development build because it contains native code. MapLibre

That's not a major issue for a proper project.

Your UI might eventually look like:

```
SearchLocation screen
        ↓
RoutePreferences screen
        ↓
POST /routes/recommend
        ↓
RouteResults screen
 ├─ Route 1: Recommended
 ├─ Route 2: Most Accessible
 └─ Route 3: Fastest
        ↓
Map / Navigation screen
```

2. Backend — FastAPI

Given your team's preference for Python, I would choose **FastAPI**.

I would pick it over Flask for this project because you're building an API-centric application rather than a traditional server-rendered website.

FastAPI is designed specifically around Python APIs, supports Python type hints and request validation, generates interactive OpenAPI documentation automatically, and supports asynchronous operations well. FastAPI

That last part is particularly useful because your backend will frequently be doing things like:

```
Receive request
      ↓
Call OneMap Search API
      ↓
Call OneMap Routing API
      ↓
Query accessibility/shade data
      ↓
Calculate route scores
      ↓
Rank routes
      ↓
Return JSON
```

So your backend stack could be:

```
Python
FastAPI
Pydantic
HTTPX
SQLAlchemy 2.x
GeoAlchemy2
Uvicorn
```

FastAPI

Handles endpoints such as:

```
POST /routes/recommend

GET /locations/search

GET /routes/{route_id}

GET /users/{id}/mobility-profile
PUT /users/{id}/mobility-profile

POST /hazards
```

Pydantic

FastAPI uses Pydantic models to validate incoming/outgoing data.

For example:

```
class RouteRequest(BaseModel):
    start_lat: float
start_lon: float

    end_lat: float
end_lon: float

    wheelchair_accessible: bool
prefer_shade: bool
avoid_stairs: bool
```

This is very convenient for a student team because malformed API requests are caught automatically rather than requiring you to write tons of manual validation.

HTTPX

Use **HTTPX** for requests from your backend to OneMap.

For example conceptually:

```
React Native
     |
     | POST /routes/recommend
     ▼
FastAPI
     |
     | HTTP request
     ▼
OneMap API
```

Don't call OneMap directly from the mobile app.

Keep your OneMap credentials/token handling in the backend.

OneMap's routing endpoints require an authorization token, and OneMap explicitly recommends securely storing credentials rather than exposing them. Their current tokens are valid for three days. OneMap

That means:

```
BAD
```

```
Mobile App ──────── OneMap
        API credentials exposed


GOOD

Mobile App ─── FastAPI ─── OneMap
│              credentials stored
server-side
```

3. Database — PostgreSQL + PostGIS

For your application, I would **strongly recommend PostgreSQL rather than MongoDB/Firebase**.

And specifically:

PostgreSQL + PostGIS

PostGIS is a PostgreSQL extension specifically designed for geographic data. It adds spatial data types, spatial indexing, distance calculations, intersections, coordinate operations, GeoJSON support, etc. PostGIS That fits your application extremely well.

You could store ordinary information such as:

```
users mobility_profiles route_preferences
saved_routes reports
```

alongside geographic information such as:

```
shaded_walkways
sheltered_walkways
stairs ramps lifts
hazards
accessible_segment
s pedestrian_paths
```

For example:

```
users -------------
-----id email name


mobility_profiles ----
--------------user_id
wheelchair_user
avoid_stairs
prefer_shelter
max_walking_distance
```

```
map_segments ------
------------id
geometry
is_sheltered
is_accessible
has_stairs gradient
surface_type
```

The `geometry` column could contain an actual geographic line:

```
LINESTRING(
    103.6812 1.3456,
    103.6815 1.3459,
...
)
```

PostGIS can then efficiently answer questions such as:

> Which sheltered paths intersect this candidate route?

or:

> How much of this route passes through sheltered areas?

or:

> Are there stairs within 5 metres of this route?

That's where PostGIS becomes much more useful than a normal database. Its spatial indexes are specifically designed to speed up this sort of geographic query. PostGIS

Use **SQLAlchemy** as your Python ORM between FastAPI and PostgreSQL. The current stable SQLAlchemy 2.0 series supports PostgreSQL and both synchronous and asynchronous database access. SQLAlchemy Doc... +1 So:

```
FastAPI
   |
SQLAlchemy
   |
PostgreSQL
   |
PostGIS
```

4. Your route recommendation engine

I'd keep the actual recommendation logic **inside the Python backend**.

Don't create a separate ML service or AI component. You don't need one initially.

Something like:

```
User enters:

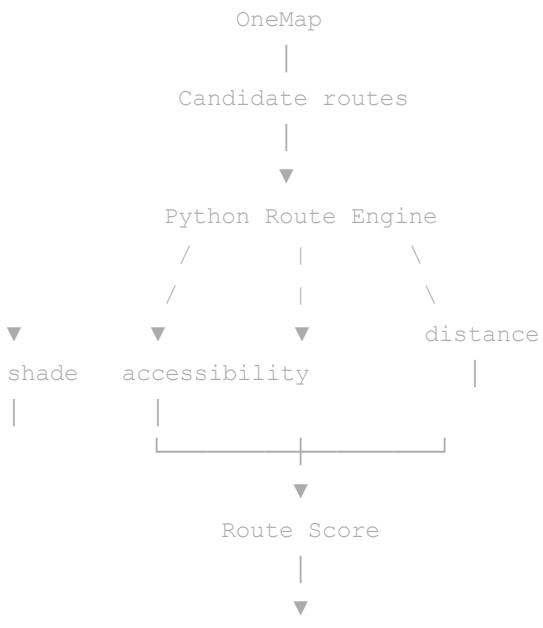
NTU
 ↓
Jurong Point

Preferences:
  ✓ Prefer sheltered paths
  ✓ Avoid stairs
  ✓ Wheelchair accessible
```

FastAPI receives:

```
{
  "origin": "...",
  "destination": "...",
  "preferences": {
      "shade": true,
      "wheelchair": true,
      "avoid_stairs": true
  }
}
```

Then:



```
Rank routes
```

For example, internally:

```
score = (
    0.30 * travel_time_score
    + 0.30 * accessibility_score
    + 0.25 * shade_score
    + 0.15 * distance_score
)
```

The actual weights would change depending on the user's mobility profile.

A wheelchair user's scoring function might instead effectively be:

```
Accessibility    50%
Travel time      20%
Shade            15%
Distance         15%
```

whereas an ordinary pedestrian who selects **"Prefer Shade"** might be:

```
Shade            40%
Travel time      30%
Distance         20%
Accessibility    10%
```

This ranking engine can simply be normal Python.

No machine learning is necessary.

5. OneMap fits your backend very nicely

OneMap already provides routing for:

- walking driving
- cycling public
- transport
- and returns route geometry, total distance and total time. OneMap

So you don't necessarily need to implement your own fundamental routing algorithm.

Instead, the interesting part of **your application** becomes:

```
OneMap
   ↓
Generate possible routes
   ↓
YOUR SYSTEM
   ↓
Analyse each route
   ↓
Accessibility
Shade
Shelter
Stairs Mobility
preferences etc.
↓
Rank routes
```

There is also something very relevant to your project that you should know about:

OneMap now has a Barrier-Free Access routing API.

It provides wheelchair-friendly routes designed to avoid things such as stairs, steep gradients and inaccessible paths. However, the API is currently available only to approved users and currently covers selected parts of Singapore rather than the entire country. OneMap

If this is your NTU accessibility routing project, I would **definitely contact SLA about getting access**.

You could then eventually have:

```
Regular OneMap routing
        +
OneMap Barrier-Free routing
        +
Your sheltered/shaded-path information
        +
User Mobility Profile
        ↓
Your recommendation engine
```

That would be a much stronger architecture than trying to recreate accessibility routing entirely yourselves.

The stack I would actually submit

If this is for your SWE project and you need to explicitly state your technology stack, I'd keep it this clean:

Component	Technology
Mobile frontend	**React Native**
Frontend language	**TypeScript**
Mobile framework/tooling	**Expo**
Map rendering	**MapLibre React Native**
Backend language	**Python**
Backend framework	**FastAPI**
API communication	**REST / JSON + HTTPX**
Data validation	**Pydantic**
ORM	**SQLAlchemy 2.0**

Component	Technology
Database	**PostgreSQL**
Geospatial extension	**PostGIS**
External map/routing API	**OneMap API**
Authentication	JWT-based auth / managed auth later
Deployment	Managed Python server + managed PostgreSQL

In one sentence

> **React Native + TypeScript + Expo** for the mobile frontend, **Python + FastAPI** for the backend and route-ranking engine, and **PostgreSQL + PostGIS** for user and geospatial data, with the backend integrating with **OneMap's routing and accessibility APIs**.

For the size of the application you're describing, I think this gives you a particularly good balance: **not overly complex for a university team, but architecturally strong enough that you wouldn't have to throw it away if the application became considerably larger later.**



ChatGPT can make mistakes. Check important info.

(3) FastAPI is appropriate for our project because our team is more comfortable using Python, so it allows us to build the backend API without having to learn a completely new language or framework. It is also lightweight and relatively simple to set up, which makes it suitable for a course project where development time is limited and we do not need the complexity of a large-scale commercial backend.